

Name:

Matrikel-Nr:

Klausur

Softwareentwicklung mit JavaScript

Wintersemester 2023–2024

28.02.2024

Hinweise:

- Bearbeitungszeit: **90** Minuten.
- Zugelassene Hilfsmittel: schriftliche Unterlage.
- Eine 100% Leistung (Note 1.0) entspricht mindestens 25 Punkten. Man kann höchstens 30 Punkten erreichen.

Viel Erfolg!

[illegible]

Verständnisse Fragen

Aufgabe 1.

2 P.

Der Browser lädt keine JavaScript-Datei `myscript.js` herunter. Der Status-Code des HTTP-Response ist 404 (Not found). Nennen Sie zwei mögliche Fehlerquelle.

Aufgabe 2.

1 P.

Nennen Sie ein Programm, um 3rd Party Bibliotheken in einem JavaScript Projekt zu verwalten.

Aufgabe 3.

2 P.

Sind folgende Funktionen *pure function*? Begründen Sie Ihre Behauptung.

1. Die Funktion

```
function setValue(v) {  
  document.getElementById("name").value = "" + v;  
}
```

Das Argument `v` ist ein beliebiges Objekt.

2. Die Funktion

```
function accumulate(data) {  
  let acc = [];  
  acc[0] = data[0]  
  for(let i = 1; i < data.length, i++) {  
    acc[i] = acc[i-1] + data[i];  
  }  
  return acc;  
}
```

Das Argument `data` ist ein Array von Datentype `Number`.

Aufgabe 4.

1 P.

Vervollständigen Sie die Dokumentation der Funktion `accumulate` unten:

Zeit und Datum

Gegeben ist das Modul `chronos.js` wie folgt:

```
/**
 * checks if a year is a leap year.
 *
 * @param year {number} a valid year in gregorian calendar
 *
 * @return {boolean} true for a leaf year, false otherwise
 * */
export function isLeapYear(year) {
  let leapYear = (year % 4 === 0);
  leapYear = leapYear && (year % 100 !== 0);
  leapYear = leapYear || (year % 400 === 0);
  return leapYear;
}

/**
 * finds the weekday of a given date by a tuple of (day, month, year).
 *
 * @param {number} day of month (first day in a month is always 1)
 * @param {number} month (1 for January and 12 for December)
 * @param {number} year (a year in Gregorian calendar)
 *
 * @return {number} a number from 0 (Sunday) to 6 (Saturday)
 * */
export function weekDayOfDate(day, month, year) {
  let y0 = year - ((14-month)/12 | 0);
  let x = y0 + ((y0/4)|0) - ((y0/100)|0) + ((y0/400)|0);
  let m0 = month + 12 * ((14-month)/12 | 0) - 2;
  return (day + x + ( (31*m0)/12 ) | 0 ) % 7;
}

/**
 * counts how many days in a month of a year.
 *
 * @param {number} month a number between 1 (for January) and 12 (for December)
 * @param {number} year a valid year in Gregorian calendar system.
 *
 * @return {number} number of days in the given month of the year (28|29|30|31).
 * */
export function countDaysInMonth(month, year) {
  const daysInMonth = [undefined,
    31, 28, 31, 30,
    31, 30, 31, 31,
    30, 31, 30, 31];
  if (month === 2 && isLeapYear(year)) {
    return 29;
  }
  return daysInMonth[month];
}
```

Aufgabe 5.

1 P.

Nennen Sie ein Programm, das die Dokument-Kommentar über jeweilige Funktionen verarbeiten kann.

Aufgabe 6.

2 P.

Wie importiert man die Funktion `isLeapYear` in der Datei `calendar.js` in dem gleichen Ordner wie die Datei `chronos.js`, damit die Funktion `isLeapYear` in der Datei `calendar.js` genutzt werden kann.

Aufgabe 7.

6 P.

Schreiben Sie eine Funktion `nextSunday(day, month, year)` in der Datei `calendar.js`. Die Funktion enthält ein Datum und gibt das Datum vom nächsten Sonntag zurück.

- Die Rückgabe ist ein Array mit drei Elementen, die ein gültiges Datum in Reihenfolge Tag, Monat, Jahr repräsentieren.
- Ist das Datum, das vom Tupel `(day, month, year)` definiert wird, ein Sonntag, gibt die Funktion das Datum des nächsten Sonntags zurück. Beispielweise gibt der Aufruf `nextSunday(19,2,2023)` den Array `[26,2,2023]` zurück.
- Sonst gibt die Funktion das Datum des nächsten Sonntag zurück. Z.B. `nextSunday(28,2,2023)` gibt `[5,3,2023]` zurück.

Benutzen Sie die Funktionen im Modul `chronos.js`. Berücksichtigen Sie die Schaltjahre, den Jahresübergang und Monatsübergang.

Sie können eine Hilf-Funktion `countDaysToSunday(day, month, year)` schreiben, um die Anzahl der Tage bis zum nächsten Sonntag zu berechnen. Die unterstehenden Abbildung zeigt einen Auszug aus einem Kalender

Dezember 2023

So	Mo	Di	Mi	Do	Fr	Sa
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

Januar 2024

So	Mo	Di	Mi	Do	Fr	Sa
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Aufgabe 8.

6 P.

Die Datei `calendar.html` im gleichen Ordner mit der Datei `calendar.js` hat folgenden HTML-Elementen:

```
<input type="text" id="day" />
<input type="text" id="month" />
<input type="text" id="year" />
<input type="button" id="calculate">Ok</input>
Nächster Sonntag: <span id="next-sunday"></span>
```

1. (1 P.) Wie wird die Datei `calendar.js` in der Datei `calendar.html` gebunden, damit die Funktionen in `calendar.js` das DOM der Datei `calendar.html` ansprechen können?
2. (3 P.) Schreiben Sie eine Funktion `showNextSunday()`, die das eingegebene Datum aus dem HTML-Elementen abliest, und den nächsten Sonntag als Text-Inhalt des Elementes `next-sunday` ausgibt. Die Ausgabe-Format ist „d.m.yyyy“ (ohne Anführungszeichen). Benutzen Sie die obige Funktion `nextSunday`.

3. (2 P.) Verbinden Sie die Funktion `showNextSunday()` mit dem Ereignis `click` des Elements `calculate`, und zwar so, damit, wenn der Button angeklickt wird, wird die Funktion (`showNextSunday`) aufgerufen.

Objekt-orientierte Programmierung

Aufgabe 9.

6 P.

Das Objekt `BigBag` speichert Items mit ihrem Gewichten. Es hat eine bestimmte Kapazität, die man setzen kann; und das aktuelle gesamte Gewicht.

Wird ein Item im `BigBag` hinzugefügt, wird zuerst geprüft, ob durch das neue Item die Kapazität des `BigBag` überschritten wird. Wird die Kapazität nicht überschritten, wird das Item in `BigBag` hinzugefügt. Das gesamte Gewicht von `BigBag` wird um das Gewicht des Item erhöht. Wird die Kapazität überschritten, wird ein Objekt vom Typ `Error` ausgeworfen. Ergänzen Sie das Objekt `BigBag`, sodass das Testprogramm funktioniert.

```
const BigBag = {
  _weight: 0,
  _capacity: 0,
  _item : [],
  set capacity /* ...Ihr Code... */,
  get items   /* ...Ihr Code... */,
  insertItem  /* ...Ihr Code... */
};

// Test Programm
// Kapazität von BigBag soll 30 Kg sein:
BigBag.capacity = 30; // → Method set capacity wird aufgerufen

// Fügt 20Kg "Sand" hinzu:
BigBag.insertItem("Sand", 20);

// Fügt 5Kg "Cement" hinzu:
BigBag.insertItem("Cement", 5);

// Fügt 10Kg "Stone" hinzu, wird ein Error ausgeworfen
try {
  BigBag.insertItem("Stone", 10);
}catch(ex) {
  console.log(ex); // Diese Zeile wird ausgeführt.
  // Die Fehlermeldung lautet: "BigBag is overloaded, maximum 30Kg"
}

let items = BigBag.items;
// items ist ein Array wie folgt:
// [ {item:"Sand", weight: 20}, {item: "Cement", weight: 5} ]
```

Aufgabe 10.

3 P.

Gegeben ist ein ungerichteter Graph G als Adjazenzliste:

```
0 : 1 → 2 → 3
1 : 0 → 2 → 4
2 : 0 → 1 → 5
3 : 0 → 4 → 5
4 : 1 → 3 → 5
5 : 2 → 3 → 4
```

- (2 P.) Zeichnen Sie den Graph und färben Sie den Graph um die chromatische Zahl $\chi(G)$ zu bestimmen.
- (1 P.) Erstellen Sie eine geeignete Datenstruktur, um den obigen Graph in JavaScript abzubilden.